

Task Title: Recipes Finder

[1]Objective:

Develop a modern, responsive, and interactive **Global Flavors – World Recipe Explorer** web application that allows users to discover, search, filter, save, and explore recipes from different countries around the world. The project aims to provide an engaging culinary experience through dynamic recipe data, intuitive navigation, personalized features, and a visually appealing user interface.

[2]Project Description:

Global Flavors – World Recipe Explorer is a premium recipe discovery platform built using HTML, CSS, and JavaScript. The application enables users to explore international cuisines and recipes through a clean, modern, and fully responsive interface. The platform features an immersive hero section, advanced recipe search functionality, country-based cuisine exploration, favorites management, recently viewed recipes tracking, dark/light theme switching, and detailed recipe viewing through interactive modal windows. Users can filter recipes by category, country, meal type, and alphabetical order, making recipe discovery fast and convenient.

Key Features:

Global Recipe Search – Users can instantly search recipes by name using real-time API-powered search functionality.

Advanced Filtering System – Recipes can be filtered by category, country, meal type, and alphabet for quick discovery.

Country-Based Exploration – Users can browse cuisines from different countries and view recipes specific to each region.

Favorites Management – Recipes can be saved to favorites using localStorage, allowing users to access them anytime.

Recently Viewed Tracking – The application stores recently viewed recipes, making it easy to revisit previously explored dishes.

Recipe Detail Modal – Detailed recipe information, including ingredients, instructions, and images, is displayed in an interactive modal window.

API-Powered Data Retrieval – Recipe content is dynamically fetched from **TheMealDB API** using the JavaScript Fetch API.

Dual Theme System – Users can switch between Light and Dark modes with smooth visual transitions across the interface.

Responsive Modern UI – Fully optimized layouts ensure a seamless experience across desktop, tablet, and mobile devices.

Interactive Statistics Dashboard – Visual counters showcase recipe, category, and country data in an engaging format.

Smooth Navigation Experience – Includes animated scrolling, back-to-top functionality, and intuitive section navigation.

Premium Visual Design – Modern typography, animations, hover effects, gradients, and responsive components create a professional user experience.

[3]Project Screenshots:

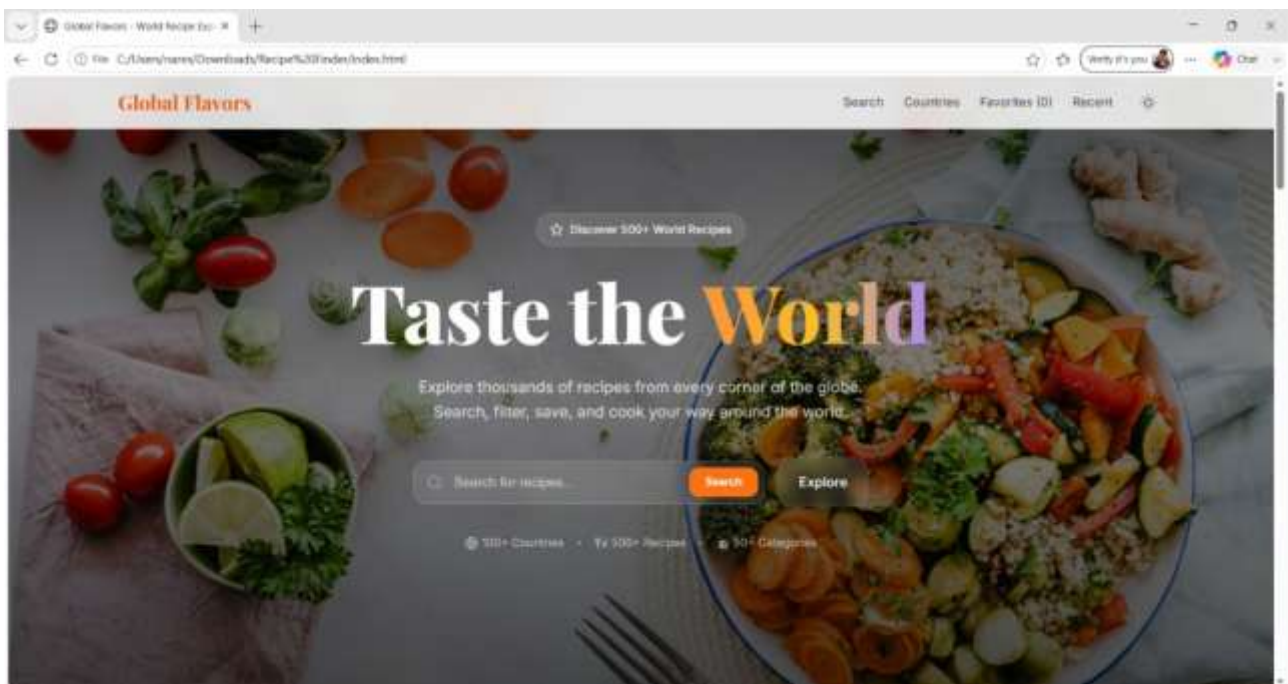


Fig 3.1: Home Page

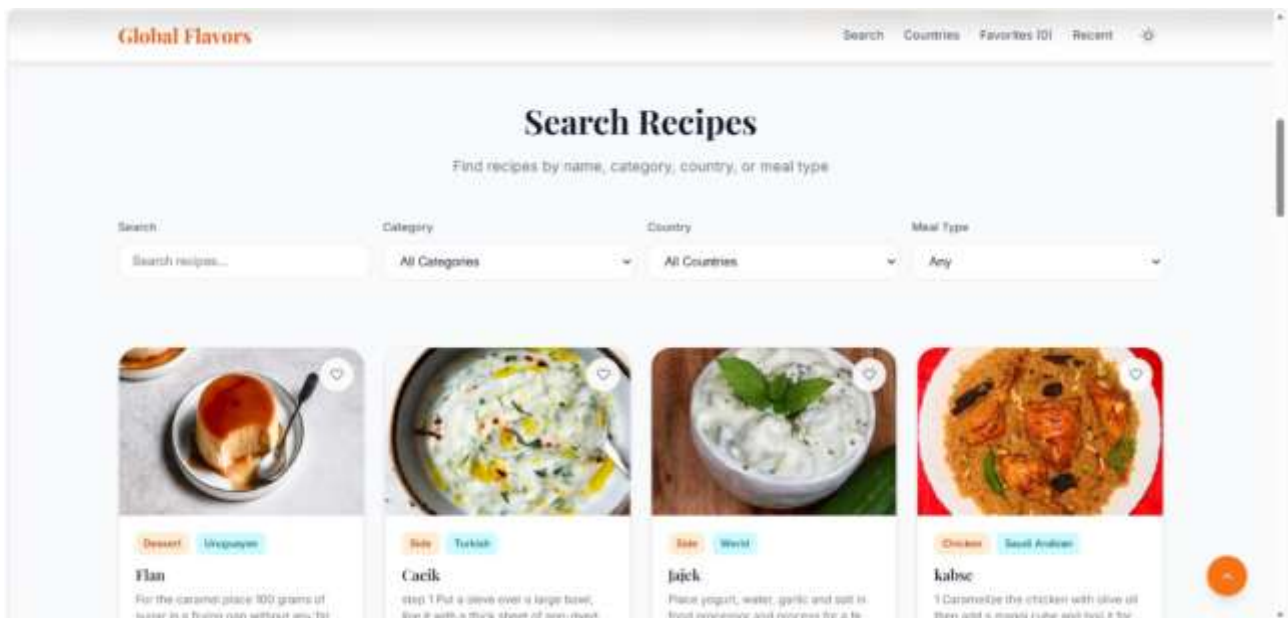


Fig 3.2: List of Recipes

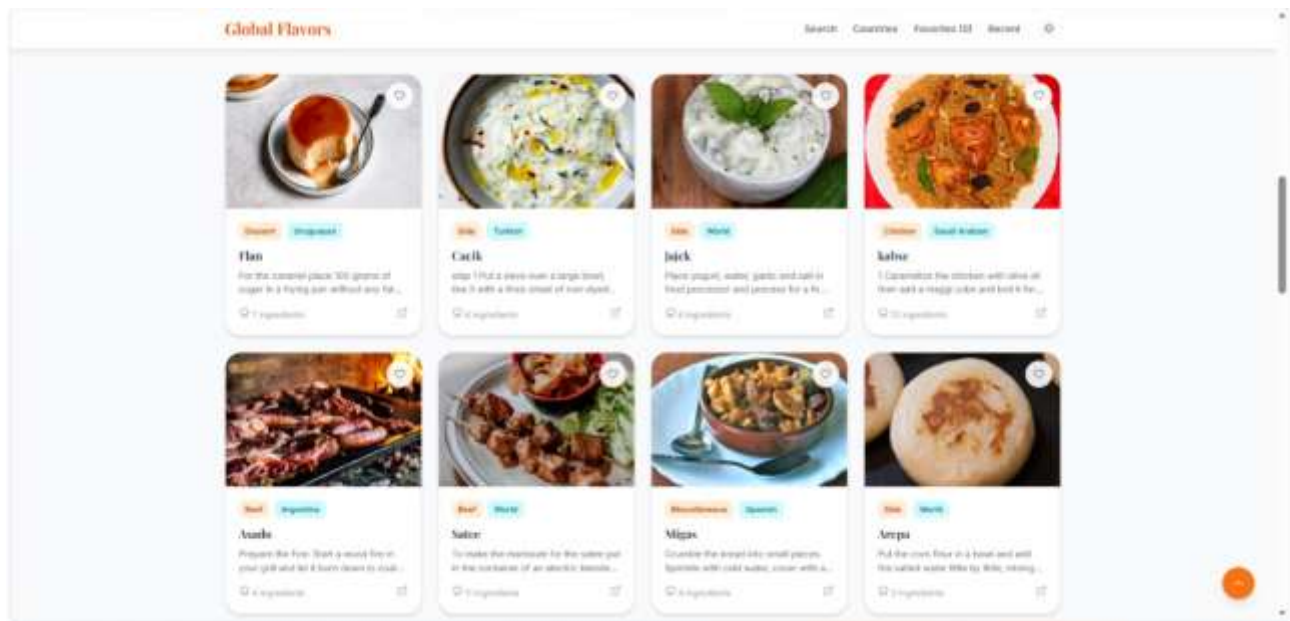


Fig 3.2(a): List of Recipes

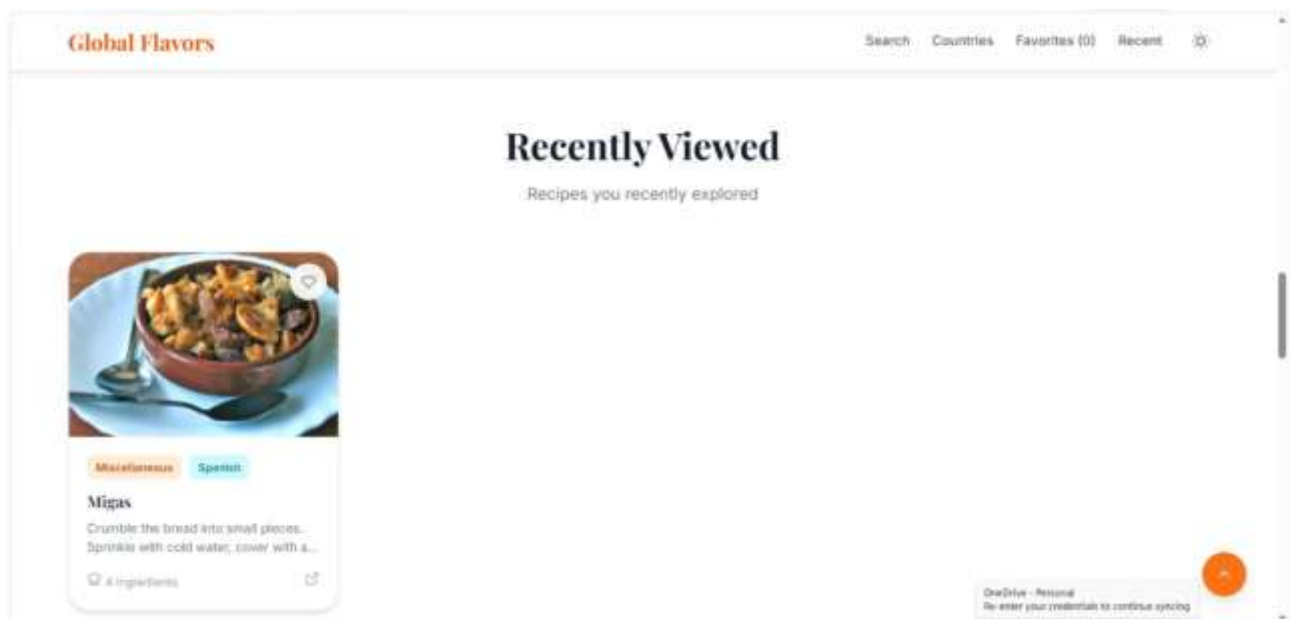


Fig 3.3: Recently Viewed Recipes

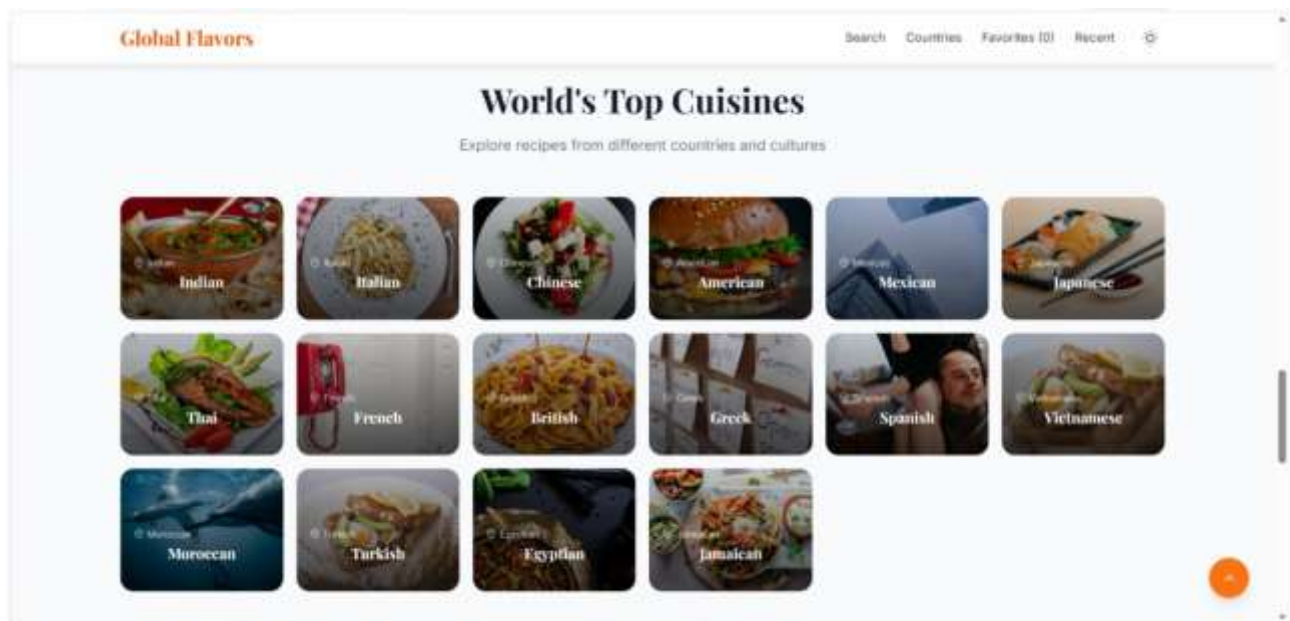


Fig 3.4: World's Top Cuisines



Fig 3.5: Favorite Recipes Will Store Here

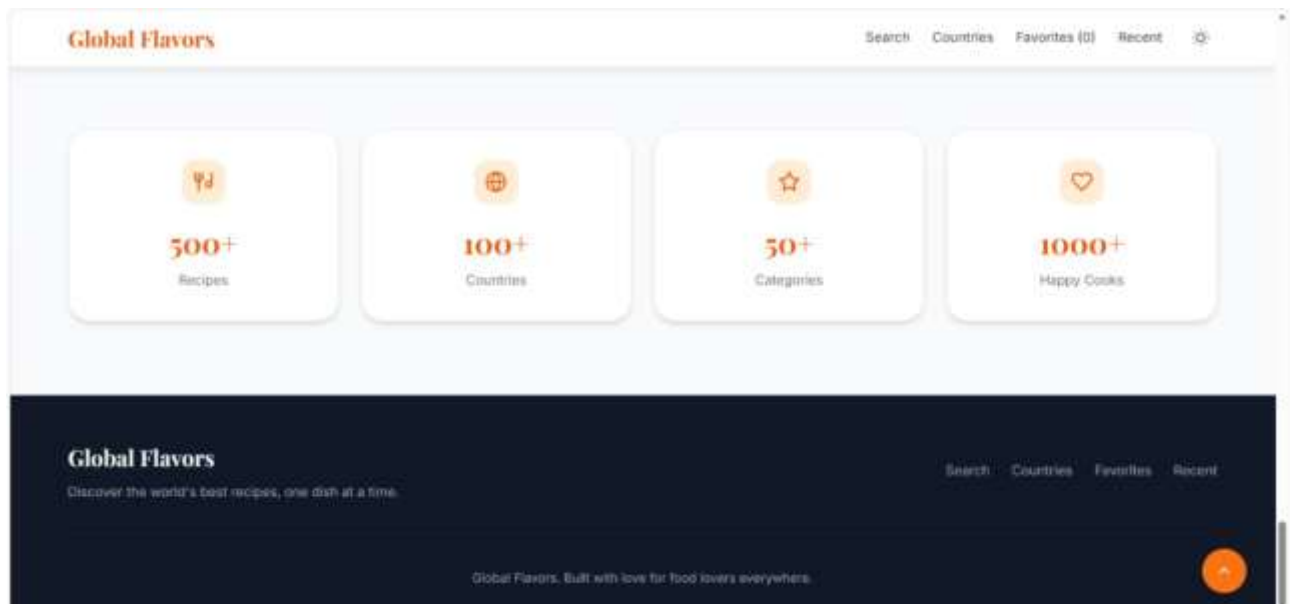


Fig 3.6: About

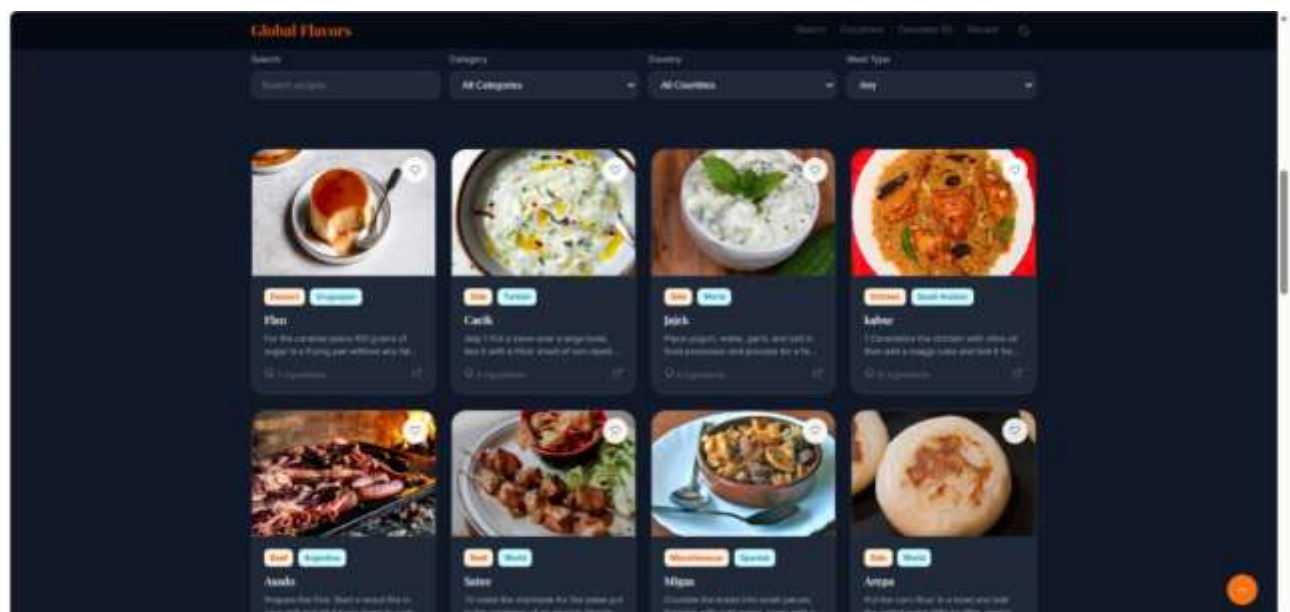


Fig 3.7: Dark Theme

[4]Code Snippets:

HTML:

```
<!-- ===== 1. PAGE SETUP + GOOGLE FONTS LINK ===== -->
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <link rel="stylesheet" href="style.css">
  <link
href="https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400;600;700;800&family=Inter:wght@300;400;500;600;700&display=swap" rel="stylesheet">
</head>

<!-- ===== 2. NAVBAR WITH THEME TOGGLE (sun/moon SVG swap) ===== -
->
<nav class="navbar" id="navbar">
  <div class="nav-container">
    <a href="#" class="nav-brand">Global Flavors</a>
    <div class="nav-links">
      <a href="#search" class="nav-link">Search</a>
      <a href="#favorites" class="nav-link">Favorites (<span
id="fav-count">0</span>)</a>
      <button id="theme-toggle" class="theme-btn" aria-
label="Toggle dark mode">
        <svg class="icon-sun" width="20" height="20" viewBox="0 0
24 24" fill="none" stroke="currentColor" stroke-width="2"><circle
cx="12" cy="12" r="5"/></svg>
        <svg class="icon-moon" width="20" height="20" viewBox="0 0
24 24" fill="none" stroke="currentColor" stroke-width="2"><path
d="M21 12.79A9 9 0 1 1 11.21 3 7 7 0 0 0 21 12.79z"/></svg>
      </button>
    </div>
  </div>
</nav>

<!-- ===== 3. HERO WITH BG IMAGE + GRADIENT TEXT + SEARCH BOX
===== -->
<section class="hero" id="hero">
  <div class="hero-bg">
    
    <div class="hero-overlay"></div>
  </div>
  <div class="hero-content">
    <h1 class="hero-title">Taste the <span class="hero-
gradient">World</span></h1>
    <div class="hero-search">
      <div class="search-box">
```

```

        <svg class="search-icon" width="20" height="20" viewBox="0
0 24 24" fill="none" stroke="currentColor" stroke-
width="2"><circle cx="11" cy="11" r="8"/></svg>
        <input type="text" id="hero-search-input"
placeholder="Search..." class="search-input">
        <button id="hero-search-btn" class="search-
btn">Search</button>
    </div>
</div>
</div>
</section>

```

```

<!-- ===== 4. FILTER BAR (search + dropdowns) – reusable for any
list page ===== -->
<div class="search-filters">
    <div class="filter-group">
        <label>Search</label>
        <input type="text" id="search-input" class="filter-input"
placeholder="Search...">
    </div>
    <div class="filter-group">
        <label>Category</label>
        <select id="category-filter" class="filter-select">
            <option value="">All Categories</option>
        </select>
    </div>
</div>
<div class="active-filters" id="active-filters"></div>

```

```

<!-- ===== 5. RESULTS GRID + LOADING + EMPTY STATES (the 3-state
pattern) ===== -->
<div class="results-grid" id="results-grid"></div>
<div class="loading" id="search-loading" style="display:none">
    <div class="spinner"></div>
    <p>Searching...</p>
</div>
<div class="no-results" id="no-results" style="display:none">
    <p>No results found.</p>
</div>
<!-- JS controls which one shows via style.display -->

```

```

<!-- ===== 6. COUNTRY/CATEGORY GRID (clickable tile -> expands
detail panel) ===== -->
<div class="country-grid" id="country-grid"></div>
<div class="country-recipes" id="country-recipes"
style="display:none">
    <div class="country-header">
        <h3 id="country-name">Recipes</h3>
        <button id="close-country" class="close-country-
btn">Close</button>

```

```
    </div>
    <div class="results-grid" id="country-recipes-grid"></div>
</div>
```

```
<!-- ===== 7. STATS SECTION (icon + number + label, repeatable
card) ===== -->
<div class="stats-grid">
  <div class="stat-card">
    <div class="stat-icon">
      <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M3 2v7..." /></svg>
    </div>
    <div class="stat-number">500+</div>
    <div class="stat-label">Recipes</div>
  </div>
  <!-- repeat .stat-card for each stat -->
</div>
```

```
<!-- ===== 8. BACK-TO-TOP BUTTON ===== -->
<button id="back-to-top" class="back-to-top" aria-label="Back to
top">
  <svg width="20" height="20" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M18 15l-6-6-6
6" /></svg>
</button>
```

```
<!-- ===== 9. MODAL SHELL (empty content div - JS injects detail
HTML) ===== -->
<div class="modal-overlay" id="modal-overlay"
style="display:none">
  <div class="modal">
    <button class="modal-close" id="modal-close" aria-
label="Close">
      <svg width="24" height="24" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2"><path d="M18 6L6 18M6 6l12
12" /></svg>
    </button>
    <div class="modal-content" id="modal-content"></div>
  </div>
</div>
<!-- JS: modal-overlay.style.display='flex' to open, 'none' to
close -->
```

```
<!-- ===== 10. FOOTER (brand + links, 2-col -> 1-col responsive)
===== -->
<footer class="footer">
  <div class="container">
    <div class="footer-content">
      <div class="footer-brand">
```



```

        <h3>Global Flavors</h3>
        <p>Tagline here.</p>
    </div>
    <div class="footer-links">
        <a href="#search">Search</a>
        <a href="#favorites">Favorites</a>
    </div>
</div>
<div class="footer-bottom"><p>© Footer note.</p></div>
</div>
</footer>

<!-- ===== 11. SVG ICON PATTERN (inline, no icon library needed)
===== -->
<svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" stroke-width="2">
    <path d="..." />
</svg>
<!-- stroke="currentColor" -> icon auto inherits text color, works
in dark mode too -->

```

Css:

```

/* ===== 1. THEME VARIABLES (root + dark mode) ===== */
:root {
    --primary-500: #f97316;
    --primary-600: #ea580c;
    --secondary-500: #8b5cf6;
    --accent-500: #06b6d4;
    --gray-50: #f9fafb; --gray-900: #111827; --gray-950: #030712;
    --font-display: 'Playfair Display', serif;
    --font-body: 'Inter', sans-serif;
    --radius: 1rem;
    --shadow: 0 4px 6px -1px rgba(0,0,0,0.1);
    --shadow-lg: 0 10px 15px -3px rgba(0,0,0,0.1);
    --transition: 0.3s ease;
}

[data-theme="dark"] {
    --bg: var(--gray-950);
    --bg-card: var(--gray-800);
    --text-primary: var(--gray-100);
    --text-secondary: var(--gray-400);
    --border: var(--gray-700);
}

/* usage: body { background: var(--bg, #fff); } -> fallback for
light mode */

/* ===== 2. STICKY GLASS NAVBAR ===== */
.navbar {
    position: fixed; top: 0; left: 0; right: 0; z-index: 100;
    background: rgba(255,255,255,0.85);
    backdrop-filter: blur(12px);
}

```

```

    border-bottom: 1px solid var(--gray-200);
}
[data-theme="dark"] .navbar { background: rgba(3,7,18,0.85); }

/* ===== 3. THEME TOGGLE ICON SWAP (no JS icon logic needed) =====
*/
.theme-btn .icon-moon { display: none; }
[data-theme="dark"] .theme-btn .icon-sun { display: none; }
[data-theme="dark"] .theme-btn .icon-moon { display: block; }
/* JS: document.documentElement.setAttribute('data-theme','dark')
*/

/* ===== 4. HERO GRADIENT TEXT ===== */
.hero-gradient {
    background: linear-gradient(to right, var(--primary-400), var(--
amber-400), var(--secondary-400));
    -webkit-background-clip: text;
    -webkit-text-fill-color: transparent;
    background-clip: text;
}

/* ===== 5. RESPONSIVE CARD GRID (mobile -> desktop) ===== */
.results-grid {
    display: grid;
    grid-template-columns: 1fr;
    gap: 1.5rem;
}
@media (min-width: 640px) { .results-grid { grid-template-
columns: repeat(2, 1fr); } }
@media (min-width: 1024px) { .results-grid { grid-template-
columns: repeat(4, 1fr); } }

/* ===== 6. CARD HOVER LIFT + IMAGE ZOOM ===== */
.recipe-card {
    border-radius: 1.5rem;
    box-shadow: var(--shadow);
    transition: transform var(--transition), box-shadow var(--
transition);
}
.recipe-card:hover {
    transform: translateY(-8px) scale(1.02);
    box-shadow: var(--shadow-xl);
}
.recipe-card-img img { transition: transform 0.5s ease; }
.recipe-card:hover .recipe-card-img img { transform: scale(1.05);
}

/* ===== 7. FAVORITE BUTTON (toggle active state) ===== */

```

```

.fav-btn {
  position: absolute; top: 0.75rem; right: 0.75rem;
  width: 2.5rem; height: 2.5rem; border-radius: 50%;
  background: rgba(255,255,255,0.9);
  color: var(--gray-500);
}
.fav-btn:hover { color: var(--red-500); background: var(--red-50); }
.fav-btn.active { background: var(--red-500); color: white; }
.fav-btn.active svg { fill: currentColor; }
/* JS: btn.classList.toggle('active') on click */

/* ===== 8. LINE-CLAMP TEXT TRUNCATION ===== */
.recipe-card-desc {
  display: -webkit-box;
  -webkit-line-clamp: 2;
  -webkit-box-orient: vertical;
  overflow: hidden;
}

/* ===== 9. LOADING SPINNER ===== */
.spinner {
  width: 40px; height: 40px;
  border: 3px solid var(--gray-200);
  border-top-color: var(--primary-500);
  border-radius: 50%;
  animation: spin 1s linear infinite;
}
@keyframes spin { to { transform: rotate(360deg); } }

/* ===== 10. MODAL (popup card detail view) ===== */
.modal-overlay {
  position: fixed; inset: 0; z-index: 200;
  background: rgba(0,0,0,0.6);
  backdrop-filter: blur(8px);
  display: flex; align-items: center; justify-content: center;
  animation: fadeIn 0.3s ease;
}
.modal {
  border-radius: 1.5rem; max-width: 800px; width: 100%;
  animation: slideUp 0.4s ease;
}
@keyframes fadeIn { from{opacity:0} to{opacity:1} }
@keyframes slideUp { from{transform:translateY(30px); opacity:0}
to{transform:translateY(0); opacity:1} }

/* ===== 11. BACK-TO-TOP BUTTON (show/hide via JS scroll listener) ===== */
.back-to-top {
  position: fixed; bottom: 2rem; right: 2rem;

```

```

width: 3rem; height: 3rem; border-radius: 50%;
opacity: 0; pointer-events: none;
transition: opacity var(--transition);
}
.back-to-top.visible { opacity: 1; pointer-events: auto; }
/* JS: window.onscroll -> btn.classList.toggle('visible',
window.scrolly > 300) */

```

```

/* ===== 12. BADGE / TAG CHIPS ===== */
.badge-category {
padding: 0.25rem 0.6rem;
background: var(--primary-100);
color: var(--primary-700);
border-radius: 0.5rem;
font-size: 0.75rem; font-weight: 600;
}

```

JavaScript:

```

/* ===== 1. API BASE + FETCH HELPER ===== */
const MEALDB = 'https://www.themealdb.com/api/json/v1/1';

async function fetchJSON(url) {
const r = await fetch(url);
if (!r.ok) throw new Error(r.statusText);
return r.json();
}
// usage: const data = await
fetchJSON(`${MEALDB}/search.php?s=chicken`);

/* ===== 2. PARSE API RESPONSE INTO CLEAN OBJECT ===== */
function parseMeal(raw) {
const ingredients = [];
for (let i = 1; i <= 20; i++) {
const name = raw[`strIngredient${i}`];
if (name && name.trim()) {
ingredients.push({ name: name.trim(), measure:
raw[`strMeasure${i}`]?.trim() || '' });
}
}
return { id: raw.idMeal, name: raw.strMeal, category:
raw.strCategory, ingredients };
}
// pattern: loop over numbered fields (Ingredient1..20) -> flatten
into array

/* ===== 3. localStorage AS DATABASE (favorites/recent) ===== */
let favorites = JSON.parse(localStorage.getItem('gf_favorites') ||
'[]');

function toggleFavorite(item) {

```

```

    const idx = favorites.findIndex(f => f.id === item.id);
    idx >= 0 ? favorites.splice(idx, 1) : favorites.push(item);
    localStorage.setItem('gf_favorites', JSON.stringify(favorites));
    renderFavorites();
}
// reusable for: saved jobs, saved internships, bookmarks --
anywhere

/* ===== 4. RECENT LIST WITH MAX-LENGTH CAP ===== */
let recent = JSON.parse(localStorage.getItem('gf_recent') ||
'[]');

function addRecent(item) {
    recent = recent.filter(r => r.id !== item.id);    // remove dupe
    recent.unshift(item);                            // add to
front
    if (recent.length > 10) recent = recent.slice(0, 10); // cap
size
    localStorage.setItem('gf_recent', JSON.stringify(recent));
}

/* ===== 5. DYNAMIC CARD CREATION (template literal -> DOM
element) ===== */
function createCard(item) {
    const div = document.createElement('div');
    div.className = 'recipe-card';
    div.innerHTML = `
        
        <h3>${item.name}</h3>
        <button class="fav-btn ${isFavorite(item.id) ? 'active' : ''}"
data-id="${item.id}">♥</button>
    `;
    div.addEventListener('click', (e) => {
        if (e.target.closest('.fav-btn')) return; // don't open modal
if fav clicked
        openModal(item);
    });
    return div;
}
// pattern: build HTML string -> attach listeners after -> avoid
event bubbling conflicts

/* ===== 6. RENDER WITH EMPTY-STATE TOGGLE ===== */
function renderList(gridId, emptyId, items) {
    const grid = document.getElementById(gridId);
    const empty = document.getElementById(emptyId);
    grid.innerHTML = '';
    if (items.length === 0) {
        grid.style.display = 'none';
    }
}

```

```

        empty.style.display = 'block';
    } else {
        grid.style.display = 'grid';
        empty.style.display = 'none';
        items.forEach(i => grid.appendChild(createCard(i)));
    }
}

/* ===== 7. DEBOUNCE (avoid firing API call on every keystroke)
===== */
function debounce(fn, ms) {
    let t;
    return (...args) => {
        clearTimeout(t);
        t = setTimeout(() => fn(...args), ms);
    };
}
// usage: searchInput.addEventListener('input',
debounce(runSearch, 400));

/* ===== 8. MULTI-FILTER SEARCH LOGIC (chain .filter() calls)
===== */
async function runSearch() {
    const query = document.getElementById('search-
input').value.trim();
    const cat = document.getElementById('category-filter').value;
    let results = query ? await searchAPI(query) : await getAll();

    if (cat) results = results.filter(r => r.category === cat);
    // chain more .filter() for each active dropdown

    renderList('results-grid', 'no-results', results);
}

/* ===== 9. MODAL OPEN/CLOSE ===== */
function openModal(item) {
    addRecent(item);
    document.getElementById('modal-content').innerHTML =
`<h2>${item.name}</h2>...`;
    document.getElementById('modal-overlay').style.display = 'flex';
    document.body.style.overflow = 'hidden'; // stop page scroll
    behind modal
}

function closeModal() {
    document.getElementById('modal-overlay').style.display = 'none';
    document.body.style.overflow = '';
}

// close on: X button, overlay click, Escape key

```

```

document.getElementById('modal-close').addEventListener('click',
closeModal);
document.getElementById('modal-overlay').addEventListener('click',
(e) => {
    if (e.target === e.currentTarget) closeModal();    // only if
clicked OUTSIDE modal box
});
document.addEventListener('keydown', (e) => { if (e.key ===
'Escape') closeModal(); });

/* ===== 10. DARK MODE TOGGLE WITH PERSISTENCE ===== */
const savedTheme = localStorage.getItem('gf_theme');
if (savedTheme === 'dark' || (!savedTheme &&
window.matchMedia('(prefers-color-scheme: dark)').matches)) {
    document.documentElement.setAttribute('data-theme', 'dark');
}
themeToggle.addEventListener('click', () => {
    const isDark = document.documentElement.getAttribute('data-
theme') === 'dark';
    document.documentElement.setAttribute('data-theme', isDark ?
'light' : 'dark');
    localStorage.setItem('gf_theme', isDark ? 'light' : 'dark');
});
// checks: user's saved choice -> else system preference -> else
default light

/* ===== 11. SCROLL-BASED UI (back-to-top + navbar shadow) =====
*/
window.addEventListener('scroll', () => {
    backToTop.classList.toggle('visible', window.scrollY > 400);
    navbar.style.boxShadow = window.scrollY > 50 ? '0 2px 10px
rgba(0,0,0,0.1)' : 'none';
});

/* ===== 12. POPULATE DROPDOWN FROM API ===== */
async function loadCategories() {
    const data = await fetchJSON(`${MEALDB}/list.php?c=list`);
    const select = document.getElementById('category-filter');
    data.meals.forEach(c => {
        const opt = document.createElement('option');
        opt.value = c.strCategory;
        opt.textContent = c.strCategory;
        select.appendChild(opt);
    });
}

/* ===== 13. SAFE INIT (handles script loading before/after DOM
ready) ===== */
function init() {

```

```
// all setup + event listeners go here
}  
if (document.readyState === 'loading') {  
  document.addEventListener('DOMContentLoaded', init);  
} else {  
  init();  
}
```

[5]GitHub Repository Link:

https://github.com/PavaniReddy-26/InternSpark_Recipe-Finder

[6]Hosted Project Link:

https://pavanireddy-26.github.io/InternSpark_Recipe-Finder/

[7]API'S Description:

1. TheMealDB API

The primary API used in the **Global Flavors – World Recipe Explorer** project is **TheMealDB API**, a free and open recipe database that provides access to recipes from various countries and categories around the world.

Purpose:

Fetch recipe information dynamically.

Search recipes by name.

Filter recipes by category.

Browse recipes by country/area.

Retrieve detailed recipe information including ingredients, instructions, images.

2. Fetch API

The browser's built-in **Fetch API** is used to communicate with TheMealDB API and retrieve recipe data asynchronously.

Purpose:

Send HTTP requests to recipe endpoints.

Receive JSON responses.

Update webpage content dynamically without reloading the page.

Overall API Integration Workflow

User enters a recipe search query or selects filters.

JavaScript sends a request using the Fetch API.

TheMealDB API processes the request and returns recipe data in JSON format.

The application dynamically displays recipe cards and details.

Users can view, save, and explore recipes without page refreshes.

This API-driven approach makes the application dynamic, scalable, and capable of providing real-time recipe information from a global recipe database.

[8]Conclusion:

Successfully developed and deployed the project using HTML, CSS, and JavaScript. The project helped in improving practical skills in web development.

Name: Linga Pavani

Candidate ID: IS-2026-12418

Domain: Frontend Developer Intern